Design Document

Project 2: IsItDown, a community outage board for public APIs
CS 5610 Web Development, Northeastern University, Fall 2026
Rishabh Tiwari and Dins Patel

Project Description

When a third party API starts misbehaving, developers lose a lot of time trying to work out whether the bug is in their own code or in the service. Big providers have status pages, but most smaller public APIs do not. IsItDown is a shared board where developers report the outages and slowdowns they are seeing, so the next person can check there first instead of debugging for an hour.

The app has two independent features. The first is an incident board, where anyone can report a problem with a service, edit or delete their report, and browse or filter what has been reported. The second is a watchlist, where a developer keeps track of the services they depend on, adds notes, and marks how they dealt with a past issue.

Both features read from one shared catalog of more than 2,000 public APIs. The catalog is read only and comes from the public APIs list on GitHub. The incident and watchlist collections are seeded with generated records so that each one has more than 1,000 documents to work with.

The app is built with Node.js and Express on the server, MongoDB through the native Node.js driver, and plain JavaScript modules in the browser. The pages are rendered on the client with fetch. We do not use Mongoose, a template engine, React, or server side rendering. It is deployed on Render, and the database is hosted on MongoDB Atlas.

User Personas

Priya, a backend developer

Priya is wiring a third party API into her service and a call keeps failing. Before she tears apart her own code, she wants to know if other developers are seeing the same failure right now. If someone has already reported it, she can stop debugging and move on.

Daniel, an on call engineer

Daniel's team depends on several outside services. He wants them all in one place, with a short note on each about what broke before and what the team did to get around it, so the next incident is faster to handle.

Sam, a developer who just hit an outage

Sam has just spent an hour on an outage that turned out to be the provider's fault. He wants to file a report with the symptoms and the start time so the next developer does not lose the same hour. Later he wants to mark it resolved, or fix a mistake in it.

User Stories

Rishabh Tiwari: Incidents collection

1. As Priya, I want to search the service catalog and see the incidents reported for a service, so I can tell whether an outage is already known.
2. As Sam, I want to file an incident report with a service, severity, status, start time and description, so I can warn other developers.
3. As Sam, I want to edit and delete my incident reports, so I can update the status or remove one that is resolved or wrong.
4. As Priya, I want to filter incidents by service and severity, so I can focus on what matters to me.

Dins Patel: Watchlist collection

1. As Daniel, I want to add services from the catalog to a watchlist, so I can keep the ones I depend on in one place.
2. As Daniel, I want to add notes and a status such as watching or worked around, so I can track how we handled past issues.
3. As Daniel, I want to edit and remove watchlist items, so the list reflects what I care about right now.
4. As Daniel, I want to browse and filter the watchlist, so I can review the services I track quickly.

Who Built What

Each of us built the full stack for our own collection: the MongoDB queries, the Express routes, the validation, and the browser code. The two features never call each other's code. They only read the shared services catalog by id.

- Rishabh: the shared setup (server, database connection module, services catalog and its seed script, landing page and shared navigation), plus the incidents collection with its routes, seed script and incidents page.
- Dins: the watchlist collection with its routes, seed script and watchlist page.

One design decision worth stating: there are no user accounts, so the watchlist is a single shared list instead of one list per person. We kept it simple so both halves could be finished and fully working in the time we had.

Design Mockups

These are screenshots of the three pages as built. Each page uses the same navigation bar at the top, so moving between the landing page, the incident board and the watchlist feels like one app.

Landing page

A short headline, two large links to the incident board and the watchlist, a list of the five newest incidents, and a line showing how many services and incidents are tracked.

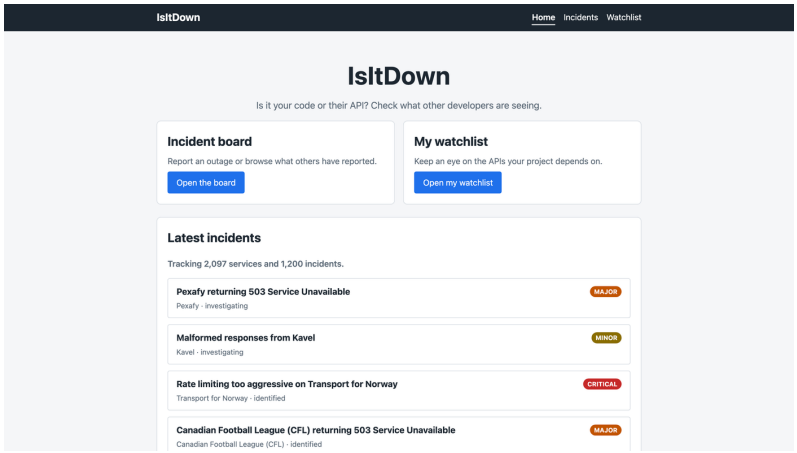


Figure 1. Landing page.

Incident board

A form to report an incident, a service search box, and below it the list of incidents with severity and service filters, edit and delete buttons, and page controls.

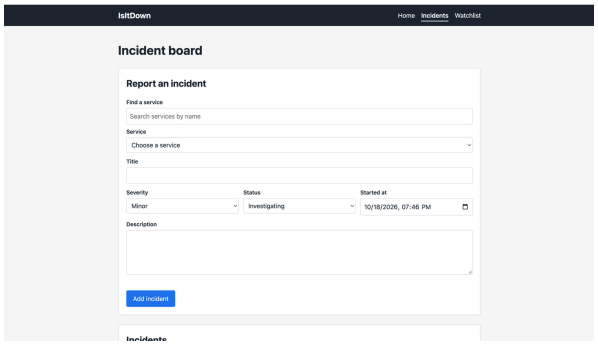


Figure 2. Incident board, report form.

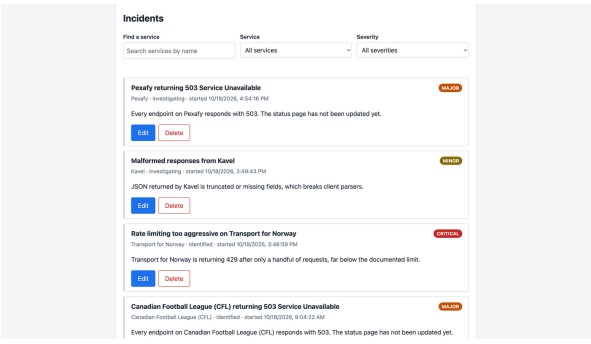


Figure 3. Incident board, list with filters, edit and delete.

Watchlist

A service search and picker, a form to add a watched service with a status of watching or worked around plus notes, and a list of watched services that can be filtered by service and status, with edit and delete buttons.

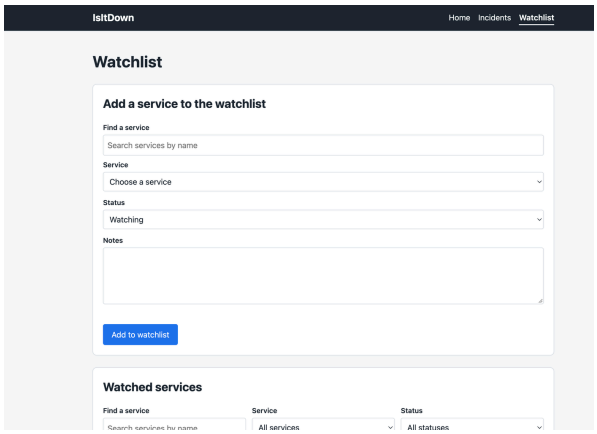


Figure 4. Watchlist, add form.

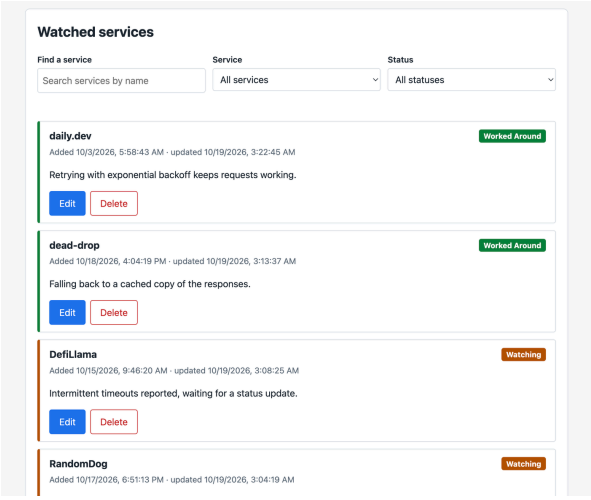


Figure 5. Watchlist, watched services with filters, edit and delete.